面向对象程序设计

类与对象



阅读例5-1，并思考相关问题

p5_1_a.cpp

Miscellaneous Fil | (Global Scope)

```
7   struct  Clock {
8       int H,M,S;
9   };
10
11  Clock MyClock;
12  void SetTime(int H,int M,int S)
13  {
14      MyClock.H=(H>=0&&H<24)?H:0;
15      MyClock.M=(M>=0&&M<60)?M:0;
16      MyClock.S=(S>=0&&S<60)?S:0;
17  }
18
19  void ShowTime()
20  {
21      cout<<MyClock.H<<":";
22      cout<<MyClock.M<<":";
23      cout<<MyClock.S<<endl;
24  }
25
26  int main(void)
27  {
28      ShowTime();
29      SetTime(8,30,30);
30      ShowTime();
31      return 0;
32  }
33
```

p5_1_b.cpp

Miscellaneous Files | Clock

```
7
8   class Clock{
9   private:
10      int H,M,S;
11  public:
12      void SetTime(int h,int m,int s)
13      {
14          H=(h>=0&&h<24)?h:0;
15          M=(m>=0&&m<60)?m:0;
16          S=(s>=0&&s<60)?s:0;
17      }
18      void ShowTime()
19      {
20          cout<<H<<":"<<M<<":"<<S<<endl;
21      }
22  };
23
24  int main(void)
25  {
26      Clock MyClock;
27      MyClock.ShowTime();
28      MyClock.SetTime(8,30,30);
29      MyClock.ShowTime();
30      return 0;
31  }
32
```

类 —— 封装

在C++中，我们是采用类（class）的形式来实现封装的。

```
class Student                       //类（class）
{
    string stu_name;                //学生姓名
    string stu_code;                //学号
    double str_score[4];            //学生4门功课成绩
public: //接口
    void set_stuInfo();             //输入学生信息
    int count_NoPass();             //计算成绩不及格门数
    double total_Score();           //计算总成绩
    bool is_better();               //判定成绩是否优秀
};
```

private：隐藏

public：
对外公开、接口

类class VS 结构体struct

C 中的struct 定义

```
struct Student1
{
    string stu_name;
    string stu_code;
    double str_score[4];
};
void set_stuInfo(Student &);
int count_NoPass(const Student &);
double total_Score(const Student &);
bool is_better(const Student &);
```

C++中的class定义

```
class Student
{
    string stu_name;
    string stu_code;
    double stu_score[4];
public:
    void set_stuInfo();
    int count_NoPass();
    double total_Score();
    bool is_better();
};
```

1、struct中成员缺省为public，数据可以对外公开

2、class中成员如无设定访问属性，成员（数据成员、函数成员）缺省为私有（private），不对外公开

封装：隐藏不想公开的部分，同时提供对外访问的接口





1、三个关键词完成封装的设定（ private、 public 、 protected）

- public：公共的，谁都可以看到的
- protected：保护的，对类族非隐私，对外界隐藏的
- private：完全隐私，除了自己任何人都不知道。

2、隐藏（数据部分、代码实现部分）：通过private 和头文件（.h）实现

3、公开部分（成员函数）：提供对外访问的接口，通过 public来实现

注意访问控制属性的设定：

（1）private成员只能被类中函数所访问，对类外不可见，public成员可以被任何函数所访问，protected成员与继承有关；

（2）C++类(class)成员的缺省访问控制权限为private，结构体（struct）成员的缺省访问控制权限为public；

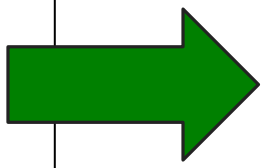
（3）通常情况下，类数据成员为private，函数成员为public；数据成员通常不为public；

（4）类中至少应提供一个外界可访问的接口（或public数据成员、或public函数成员）；

问题2：私有成员（数据成员和函数成员）如何才能被访问到呢?

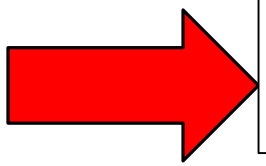
问题2：私有成员（数据成员和函数成员）如何才能被访问到呢?

```
class A  //定义类A 数据成员： 1个，成员函数：2个
{
    int a;
public:
    void init(int aa) {a=aa;}
    int getdata() {return a;}
};                  //注意分号不可少
```



有对外接口

```
class B //定义类B
{//数据成员：2个，都为私有，不可访问
    int a;
    int b;
}bx;            //声明的同时实例化bx为B类的对象
```



无对外接口

三、类的成员函数定义

成员函数是对类中封装的数据进行操作的唯一途径，有两种模式

1、内联模式：成员函数的定义在类内实现。

成员函数实现格式：

返回值 函数成员名（参数表）

{ 函数体

}

定义在类声明中的函数自动成为内联函数（ inline ），通常将函数体执行语句行数不多的成员函数设置为内联函数。

```
class Student
{
public:
        string stu_name;//学生姓名
        string stu_code;//学号
private:
        double stu_score[4];//学生4门功课成绩
public:
//（1）从主调函数接收实参设置/修改学生姓名、 学号
        void set_stuInfo(string name,string code){
        stu_name=name;
        stu_code=code;
        }
//…  代码未完，参考上面示例
};
```

内联模式定义

三、类的成员函数定义

2、类外定义成员函数：声明放在类定义的主体中，而函数的具体实现，通常放在类定义外面。

成员函数实现格式：

::域作用分辨符(表征实现的成员函数是属于某个类的)

```
返回值类型  类名::函数成员名（参数表）
{
    函数体
}
```

1、Student.h文件

```
#ifndef STUDENT_H
#define STUDENT_H
class Student
{
public:
    string stu_name;
    string stu_code;
private:
    double stu_score[4];
protected:
    int count_NoPass();
    double total_Score();
public:
    void set_stuInfo();
    bool is_better();
};
#endif // STUDENT_H
```

函数声明

2、Student.cpp文件

类外函数定义

```
void Student::set_stuInfo()
{
    cin>>this->stu_name>>this->stu_code;
    int i=0;
    for(i=0;i<4;i++)
        cin>>this->stu_score[i];
}
inline bool Student::is_better()
{
  if(count_NoPass()||total_Score()/4<85)
      return false;
  else
      return true;
}
```

类外内联函数定义

问答题3:（1）count_NoPass()//补充成员函数定义
（2）total_Score()//补充成员函数定义

2、类的成员函数可以是带默认值的成员函数/重载的成员函数

1、带默认值的成员函数

类的成员函数的参数也可以有默认的缺省值。调用时如果不给出参数的值以默认值为准。

2、成员函数重载

2、Student.h文件

```
class Student
{
public:
    string stu_name;//学生姓名
    string stu_code;//学号
private:
    double stu_score[4];//学生4门功课成绩
public:
    //（1）在函数体内通过cin输入学生信息
    void set_stuInfo();
    //（2）从主调函数接收实参修改学生姓名、学号
    void set_stuInfo(string name,string code);
    /*（3）从主调函数接收实参修改学生第i门课的成绩，
    右边第一个参数缺省，默认为第1门课程*/
    void set_stuInfo(double score,int i=0);
```

函数重载声明

…→代码未完，参考上面示例

四、对象

对象是抽象类的实际个体，类中成员是对象具有实际的属性值和行为能力。

1、对象的创建

同定义一个一般变量：

格式：　　类名　对象名；

例如：Student XiaoMing;

这里创建了一个Student类型的对象XiaoMing

四、对象

2、对象的访问操作:　　成员运算符.

格式:　　对象名.公有成员函数名（参数表）

例:　XiaoMing.set_stuInfo();

课堂练习

P226

7.编程题（2）

定义一个点类Point，定义成员函数

Double Distance(const &Point)

求两点的距离

★构造函数与析构函数

- 以clock类为例，分析构造函数的特点

```
class Clock {
private:
        int H,M,S;
public:
        Clock(int h=0,int m=0,int s=0)//定义构造函数
        {
                H=h,M=m,S=s;
                cout<<"constructor:"<<H<<":"<<M<<":"<<S<<endl;
        }
        ~Clock()//析构函数
        {
                cout<<"destructor:"<<H<<":"<<M<<":"<<S<<endl;
        }
};
```

★构造函数与析构函数

- 阅读 5-2(P181)，了解析构函数的调用时机



★构造函数与析构函数

- 例5-3，析构函数的作用

```
class String {
~String()//析构函数
{
   if (Str!=NULL)
   {
      cout<<"End!"<<endl;
      delete [] Str;
      Str=NULL;
   }
   else
      cout<<"析构函数被调用!"<<endl;
   }
};
```

★构造函数与析构函数

概念：复制构造函数

特点：是一种特殊的构造函数，具有一般构造函数的所有特性，形参类型是本类对象的引用。

① 当用类的一个对象去初始化这个类的另一个对象时。

② 如果函数的形参是类的对象，调用函数时，进行形参和实参结合时。

③ 函数的返回值是类的对象，执行完成时将返回值返回。

★构造函数与析构函数

复制构造函数调用情况

① 当用类的一个对象去初始化这个类的另一个对象时。如：

```
int main(void)
{
    Point A(1,2);
    //调用复制构造函数:用Point类的一个对象，即A来初始化这个类的另一对象B
    Point B(A);
    cout<<B.GetX()<<endl;
    return 0;
}
```

★构造函数与析构函数

复制构造函数调用情况

② 如果函数的形参是类的对象，调用函数时，进行形参和实参结合时。

```
void f(Point p)
{
        cout<<p.GetX()<<endl;
}
int main()
{
        Point A(1,2);
        f(A);
        return 0;
}
```

函数f的形参为Point类的对象，当调用函数f时，复制构造函数被调用，生成了Point类对象A的副本。

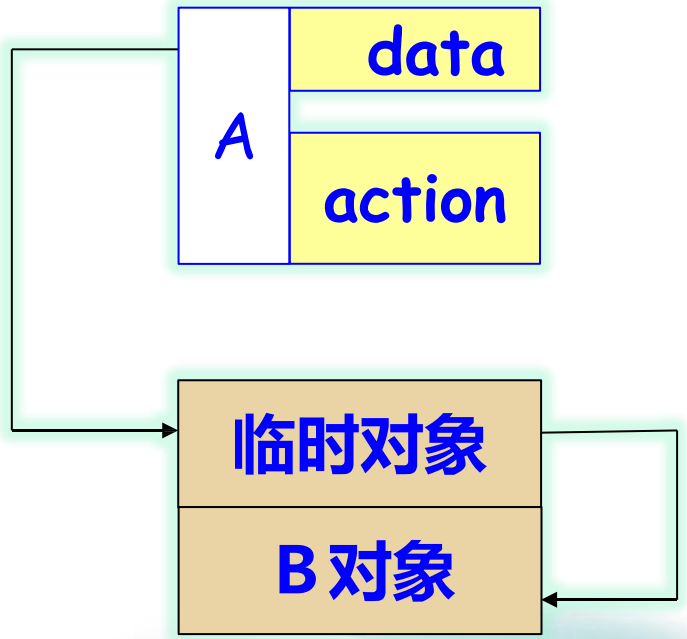
★构造函数与析构函数

复制构造函数调用情况

③ 函数的返回值是类的对象，函数执行完成返回调用者时。

注意： Visual C++环境支持；但codeblock、QT、Dev C++不支持这样操作。

```
Point g()
{
    Point A(1,2);
    return A;        //函数返回值为point类对象，
    //所以返回函数值时，调用复制构造函数
}
int main()
{
    Point B;
    B=g();  //生成A的副本给B
    return 0;
}
```



★构造函数与析构函数

复制构造函数调用情况

④ 同类对象实例赋值时，始终调用系统默认拷贝构造函数。

```
Point c3 = c1;
```

⑤ 临时对象实例用于初始化/传值/返回值/赋值(①~④情形)时，始终调用系统默认拷贝构造函数（支撑环境Visual C++）。

```
Point c4(Point (3,3));
Point c5= Point (2,3);
func(Point (2,3));
```

★对象声明及使用

1. 对象数组

定义形式

① 对象数组中所有元素都是对象，每一个元素对象都具有数据成员和函数成员。

类名 数组名 [下标表达式]

```
Point t1[2][3];
```

② 与基本类型数组一样，在使用对象数组时也只能引用单个数组元素（也就是对象）。

数组名[下标].成员名

```
t1[1][0].X;
t1[1][0].GetX();
```

访问一般形式

★对象声明及使用

对象数组

③ 对象数组的初始化：对象数组的初始化过程，实际上是每一个元素对象调用构造函数的过程。有两种情况：

- 定义数组时给每一个数组元素指定显式的初始值，则调用相应有形参的构造函数

```
Point  p1[2]={Point(1,2),Point(2,1)};
```

- 如果没有指定初始值，则调用默认构造函数

```
Point p2[2]={Point(2,2)};
```

思考：没有默认构造函数会有什么问题?

★对象声明及使用

2. 对象指针

对于对象的访问，既可以通过**对象名**，也可以通过**对象地址**来访问一个对象。**对象指针**就是用来**存放对象地址的变量**。

① 对象指针声明：**类名 *对象指针名;**

```
Point *p_point; //指针变量声明
Point myPoint;
p_point =&myPoint;//赋值要用&取得对象
        的地址
```

② 对象指针调用成员：**对象指针名->成员名**

```
p_point->GetX();
```

★对象声明及使用

对象指针

④ new动态分配对象空间，调用构造函数完成(注意：语法中的[]表示可以有，可以无)

```
Point  *p=new Point (20,20);
p->print();
```

★对象声明及使用

⑤ 用delete释放堆对象，调用的是析构函数

```
Point *p=new Point (20,20);
Point *p1=new Point [20];
Point (*p2)[5]=new Point [4][5];
delete p;
delete []p1;
delete []p2;
```

★对象声明及使用--this指针

3. OOP中的this指针对象

编译环境提供了一个名为this 的 指针，用来指向不同的对象，成员函数访问关键就在this指针。编译器会自动将对象本身的地址作为一个隐含参数传递给函数。

一个对象的this指针并不是对象本身的一部分，不影响sizeof(对象)结果。

★对象声明及使用--this指针


```
class Point
{
    int x;
    int y;
 public:
    Point():x(0),y(0){}
    void fun(int i,int j)
    {
        x = i,y=j;
    }
};
```

```
//其中，非静态成员函数fun在编译器看来应该是：
void fun(int i,int j, Point * const this)
{
    this->x = i; this->y = j;
}
```

- this在成员函数的开始前构造，在成员的结束后清除。

当调用一个类的成员函数时，编译器将类的指针作为函数的this参数传递进去。如：

```
Point a;
a.fun(5,5);
```

此处，编译器将会编译成：

```
a.fun(5, 5,&a);
```

系统变量，故不能求其地址或对其赋值。

非静态成员函数中可以直接使用this来代表指向该函数作用的对象的指针。

★对象声明及使用--this指针

① this指针是一个隐含于每一个成员函数中的特殊指针。它是一个指向正在被该成员函数操作的对象，也就是要操作该成员函数的对象。

② this作用域在类内部，当对一个对象调用成员函数时，编译程序先将对象的地址赋给this指针，编译器会自动将对象本身的地址作为一个隐含参数传递给函数。

③ 即使你没有写this指针，编译器在编译的时候也是加上this的，它作为非静态成员函数的隐含形参。被调用的成员函数函数体内所有对类成员的访问，都会被转化为this->类成员的方式。

★对象声明及使用--this指针


```
class Point
{
    int x;
    int y;
 public:
    Point():x(0),y(0){}
    void fun(int i,int j)
    {
        x = i,y=j;
    }
};
```

```
//其中，非静态成员函数fun在编译器看来应该是：
void fun(int i,int j, Point * const this)
{
    this->x = i; this->y = j;
}
```

- this在成员函数的开始前构造，在成员的结束后清除。

当调用一个类的成员函数时，编译器将类的指针作为函数的this参数传递进去。如：

```
Point a;
a.fun(5,5);
```

此处，编译器将会编译成：

```
a.fun(5, 5,&a);
```

系统变量，故不能求其地址或对其赋值。

非静态成员函数中可以直接使用this来代表指向该函数作用的对象的指针。

★对象声明及使用--this指针

以下场景中，经常需要显式引用this指针：

① 在类的非静态成员函数中返回类对象本身时，直接使用 return *this，如：实现对象的链式引用。

② 当参数与成员变量名相同时，如 this->x = x，不建议写成 x = x。

③ 避免对同一对象进行赋值操作。

但是：

静态成员函数中不能使用 this 指针！

因为静态成员函数并不具体作用于某个对象！

记住：静态成员函数的真实的参数个数，就是程序中写出的参数个数！

★类的非静态成员和静态成员static


```
class Student
{
    string stu_name;//学生姓名
    string stu_code;//学号
    double stu_score[4];//学生4门功课成绩
public:
     Student(string name1,string  code1);
};
```

实例属性：非静态成员

问题：如何统计学生对象总数?

- 用类外的变量存储总数，不能实现数据的隐藏
- 若在类中增加一个数据成员来存放总数，那每个对象都会存储一个副本，就会产生冗余，且每个对象分别维护一个“总数”，将造成数据的不一致性。
- 面向对象的类属性：使用类的静态成员，对象与对象共享数据内容



★类的静态成员static

1. 静态数据成员

- 类的普通数据成员：在每个对象中都拥有一个独立空间，即每个对象的同名数据成员可以分别存储不同数值；
- 类的静态数据成员：以static关键字来声明，这个类的所有对象共同维护使用这个静态数据成员，从而实现同一类的不同对象之间的数据共享，静态生命周期。

静态数据成员的声明

- 在类的声明中对静态数据成员进行引用性说明（类内声明）
- 必须在.CPP作用域的某个地方使用类名限定符::进行定义性说明，同时可以进行初始化。（类外进行初始化）
- 如果未初始化，系统缺省值为0（不同编译器不同，最好都要初始化）

★类的静态成员static


```
class point
{
public:
    point(int xx=0,int yy=0)
    {x=xx;y=yy;countP++;}
    point(point &p ) ;
    int getX() {return x;}
    int getY() {return y;}
    static void getC() //静态成员函数
    {cout<<"object:"<<countP<<endl;}
    static int countP;//静态数据成员
private:
    int x,y;
};
point::point(point &p) //拷贝构造函数
{
   x=p.x;  y=p.y;  countP++;
}
```

```
int point::countP=0; //静态数据成员类外初始化
int main()
{
   point A(4,5);
   cout<<"A:"<< point::countP <<endl;
   point B(A);
   cout<<"B:"<< B.countP <<endl;//借用B的类型point
   B.getC(); return 0;
}
```

① 声明：类内

② 初始化：类外cpp实现，按类外实现模式

数据类型 类名::成员名=初值;

③ 使用：类的所有对象共享 类名::成员名

④ sizeof(对象)大小：跟实际对象无关，被对象群体共享

静态数据成员

★类的静态成员static

2. 静态成员函数

优点 不依赖于任何对象(不含this指针)，可以直接访问静态数据。

① 定义：static 成员函数，可以访问静态数据

```
static void getC() //静态成员函数
{ cout<<"object: "<<countP<<endl; }
```

② 调用静态成员函数

访问方式有两种 (1) 对象名.静态成员函数 (2) 类名::静态成员函数

```
int main()
{
    point A(4,5);
    cout<<"Point A:"<<A.getX()<<","<<A.getY()<<endl;
    A.getC(); // (1) 用对象名来引用：但不访问操作A内的对象，只是借用A的类型point
    point B(A);
    cout<<"Point B:"<<B.getX()<<","<<B.getY()<<endl;
    point::getC();// (2) 用类名引用：输出对象编号countP,更符合场景
}
```

★类的静态成员

静态成员函数

③访问非静态数据成员

- 静态成员函数可以直接访问该类的静态数据成员和函数成员
- 若要访问非静态数据成员，则必须通过参数传递方式得到对象名，再通过对象名来访问。

```
class pointA
{
public:
    static void f(pointA a);
private:
    int x,y;
};
void pointA::f( pointA  a)
{
    cout<<x;//错误，不能直接对x进行引用
    cout<<a.x;//正确
}
```

课后

西南交通大学
Southwest Jiaotong University

平台作业

